

Parallelizing GPT-2 — “It works on my GPU”

Siggy Sigler, Sam Abderholden, Dawson Matthews

May 5, 2026

1 Paper Summary

Placeholder text.

2 Implementation

Problem Background

In this project, we implement an optimized CUDA version of the forward pass of Andrej Karpathy’s *microgpt* [1]. *Microgpt* is a minimal, dependency-free Python implementation of a GPT-style model. It is based on the GPT-2 architecture [2], with a few changes made for simplicity. Both GPT-2 and Karpathy’s *microgpt* implement:

- Decoder-only Transformer architecture
- Token + positional embeddings
- Stacked Transformer blocks
- Causal (masked) multi-head self-attention
- Residual connections around attention and MLP sublayers
- Feedforward MLP with expansion and projection
- Autoregressive next-token prediction

Since *microgpt* was intended to showcase the GPT-2 architecture in a single unoptimized Python file, the proposed model has only 4,192 parameters compared to GPT-2’s 1.6 billion parameters. In addition to the reduced model size, *microgpt* introduces several simplifications, including the use of RMSNorm instead of LayerNorm, removal of bias terms, and ReLU in place of GELU [1].

In order to keep the scope of our implementation to 10 hours, we focus only on speeding up the forward pass of *microgpt*. Since we believed that comparing a CUDA C++ version to a Python version would introduce speedups due to both our parallelization choices and language differences (C++ vs Python), we first reimplemented *microgpt* in C++, preserving the following:

- Identical layer structure
- Same ordering of operations (norm $\rightarrow$ attention $\rightarrow$ residual $\rightarrow$ norm $\rightarrow$ MLP $\rightarrow$ residual)
- Same KV caching mechanism
- Same attention computation (scaled dot-product + softmax)
- Same MLP structure (fc1 $\rightarrow$ activation $\rightarrow$ fc2)

Our C++ version is nearly identical, with differences limited to implementation details such as replacing dynamic Python data structures with pre-allocated caches, removing autograd in favor of raw floating-point computation, and expressing all operations as explicit loop-based computations over contiguous memory.

C++ Serial Baseline

Our implementation follows the training setup used in *microgpt*, which operates on a character-level dataset consisting of a set of names $\mathcal{D} = \{s^{(1)}, s^{(2)}, \dots, s^{(N)}\}$. Each name $s^{(k)} \in \mathcal{D}$ is treated as a sequence of tokens, $s^{(k)} = [t_1, t_2, \dots, t_T]$, where T denotes the sequence length. The model is trained autoregressively to predict the next token given all previous tokens. Our C++ implementation preserves this setup exactly, performing the same forward-pass computation and next-token prediction task.

Given a tokenized sequence (i.e., a single name) $[t_1, t_2, \dots, t_T]$, the model computes a sequence of logits $[y_1, y_2, \dots, y_{T-1}]$, where each y_i represents the model’s prediction of token t_{i+1} conditioned on the prefix $[t_1, \dots, t_i]$. In the baseline implementation, this computation is performed sequentially, iterating over the dataset one name at a time and, within each name, one token at a time, with each position attending over all previous tokens in the sequence. A high level pseudocode of our serial implementation is shown under Algorithm 1.

We denote the embedding dimension as d , the number of Transformer layers as L , and the number of attention heads as H . For each token position i , the model produces a hidden representation $x_i \in \mathbb{R}^d$, which is transformed through attention and MLP operations across all layers.

The computational cost of the forward pass over N sequences is $O(N \cdot L \cdot (C_{\text{attn}} + C_{\text{mlp}}))$, where $C_{\text{attn}} = O(T^2 \cdot d)$ and $C_{\text{mlp}} = O(T \cdot d^2)$. Since the names have approximately constant sequence lengths, we can simplify this to $O(N \cdot L \cdot d^2)$.

Algorithm 1 High-Level Forward Pass

```

loss  $\leftarrow$  0
for each sequence  $s^{(k)} \in \mathcal{D}$  do
  for  $i = 1$  to  $T - 1$  do
     $x_i \leftarrow$  TokenEmbedding( $t_i$ ) + PositionalEmbedding( $i$ )
     $x_i \leftarrow$  RMSNorm( $x_i$ )
    for  $\ell = 1$  to  $L$  do
       $x_i \leftarrow x_i +$  Attention(RMSNorm( $x_i$ ),  $\{k_j, v_j\}_{j \leq i}$ )
       $x_i \leftarrow x_i +$  MLP(RMSNorm( $x_i$ ))
    end for
     $y_i \leftarrow$  Linear( $x_i$ )
    loss  $\leftarrow$  loss  $-$  log(Softmax( $y_i$ )[ $t_{i+1}$ ])
  end for
end for
loss  $\leftarrow$  loss/total_tokens

```

3 Parallelization Strategy

Using the *perf time* command we were able to get high level timing characteristics of the C++ baseline program, especially how much time our C++ baseline was performing OS overhead versus useful compute work. With nearly zero system time (0.008s out of 2.74s total) and only 269 context switches, the program runs almost entirely in userspace with negligible OS overhead. An IPC of 3.06 with 0.999 CPUs utilized confirms the single core is saturated with compute for the full duration. Next, we used *perf record* in order to perform a call stack analysis of where the program was spending its CPU time. We found that **99.4%** of runtime is spent in linear matrix multiplications, with **70.1%** occurring in the MLP and **28.3%** in attention (note the program spends around 0.4% performing rmnsnorm, Softmax, and embedding operations). While attention is often considered the bottleneck in Transformer models, this aligns with our earlier complexity analysis, as the approximately constant sequence length causes the d^2 term to dominate, making the MLP the primary contributor to runtime.

Based on the serial workload analysis, our parallelization strategy focuses on exposing more independent work to the GPU. The original *microgpt* structure processes one name at a time and then one token at a time within that name. This creates very little parallel work per operation, especially because the names are short.

While this CPU-side analysis identifies the dominant computational patterns, it does not capture GPU-specific bottlenecks such as memory access and kernel efficiency. To guide implementation decisions at the GPU level, we additionally performed a bottleneck analysis using nvprof on an initial CUDA implementation (see Section 5). The impact of the resulting design choices is evaluated through an ablation study in that same section.

The nvprof analysis directly informed three key optimizations: introducing batching, implementing tiled matrix multiplication, switching from double to float precision. To increase available parallelism, we batch multiple names together so each kernel operates over many sequences and token positions at once, as shown in Algorithm 2. We denote the batch size as $B = |\mathcal{B}|$ and represent each batch as a tensor of shape $(|\mathcal{B}|, T, d)$. To reduce CUDA kernel complexity, we preprocess the names into batches where all sequences in a batch have the same length.

Algorithm 2 High-Level Batched Forward Pass

```
loss  $\leftarrow$  0
batches  $\leftarrow$  CreateBatches( $\mathcal{D}$ )
for each batch  $\mathcal{B} \in$  batches do
   $X \leftarrow$  TokenEmbedding( $\mathcal{B}$ ) + PositionalEmbedding(0: $T - 1$ )
   $X \leftarrow$  RMSNorm( $X$ )
  for  $\ell = 1$  to  $L$  do
     $X \leftarrow X + \text{Attention}(\text{RMSNorm}(X))$  ▷ independent per sequence
     $X \leftarrow X + \text{MLP}(\text{RMSNorm}(X))$ 
  end for
   $Y \leftarrow$  Linear( $X$ )
  loss  $\leftarrow$  loss + CrossEntropy( $Y$ , Targets( $\mathcal{B}$ ))
end for
loss  $\leftarrow$  loss/total_tokens
```

Furthermore, our strategy focuses on exposing parallelism at the level of individual tensor elements, rather than at the level of tokens or sequences. This allows each CUDA kernel to operate over $|B| \cdot T \cdot d$ elements simultaneously, significantly increasing available work per launch compared to the serial implementation.

We achieve this by decomposing the forward pass into a sequence of CUDA kernels, each operating at a finer granularity over the batched tensor. Rather than treating attention and MLP as monolithic operations, we explicitly map each sub-operation (RMSNorm, linear projections, attention, and residual additions) to separate GPU kernels. This decomposition is shown in Algorithm 3, where each step corresponds directly to a kernel launch in our implementation. Since each operation is implemented as its own kernel, we were able to reduce the granularity of the linear matrix multiplication from per-token to per-output-element, and further optimize it using tiling (as motivated by our GPU bottleneck analysis).

Algorithm 3 Layer-by-Layer GPU Forward Pass

```
for  $\ell = 1$  to  $L$  do
   $X_{\text{norm}} \leftarrow$  RMSNorm( $X$ ) ▷ Attention Block
   $Q \leftarrow$  Linear( $X_{\text{norm}}, W_Q^{(\ell)}$ )
   $K^{(\ell)} \leftarrow$  Linear( $X_{\text{norm}}, W_K^{(\ell)}$ )
   $V^{(\ell)} \leftarrow$  Linear( $X_{\text{norm}}, W_V^{(\ell)}$ )
   $A \leftarrow$  SelfAttention( $Q, K^{(\ell)}, V^{(\ell)}$ )
   $A_{\text{proj}} \leftarrow$  Linear( $A, W_O^{(\ell)}$ )
   $X_{\text{mid}} \leftarrow X + A_{\text{proj}}$ 
   $X_{\text{norm2}} \leftarrow$  RMSNorm( $X_{\text{mid}}$ ) ▷ MLP Block
   $H \leftarrow$  Linear( $X_{\text{norm2}}, W_1^{(\ell)}$ )
   $H \leftarrow$  ReLU( $H$ )
   $H_{\text{proj}} \leftarrow$  Linear( $H, W_2^{(\ell)}$ )
   $X \leftarrow X_{\text{mid}} + H_{\text{proj}}$ 
end for
```

Each operation in Algorithm 3 is implemented as a dedicated CUDA kernel, as summarized in Table 1.

From a systems perspective, the full model is transferred to the GPU once at initialization, while a reusable workspace is allocated once per batch. This avoids repeated host-device transfers and ensures that all intermediate activations remain resident on the GPU throughout the forward pass.

Lastly, the original *microgpt* implementation in Python uses FP64 precision by default, by simply changing this in our CUDA version to use floats (fp32) instead, we were able to speed up all of our compute intensive operations.

With this strategy in place, we will next evaluate how effectively the batched GPU implementation reduces wall-clock time compared to serial and PyTorch baselines.

Operation	Thread Granularity	Description
Embedding + Positional	element (B, T, d)	each thread loads one embedding value
RMSNorm	token (B, T)	each thread normalizes one token by looping over its d values
SelfAttention	element (B, T, d)	one thread computes one weighted scalar within its attn head
Linear (Q, K, V, MLP)	element $(T, \text{out_dim})$	tiled matrix mul, one thread owns one output cell
ReLU	element (B, T, d)	independent element-wise activation
Vector Add	element (B, T, d)	independent element-wise vector addition
CrossEntropy	single thread	one thread loops over all (B, T, vocab) to compute cross-entropy

Table 1: Kernel decomposition with thread granularity and execution details.

4 Performance Results with Graphs

In this section, we compare a serial C++ baseline, unbatched and batched PyTorch baselines, and our custom parallel CUDA implementation to analyze forward-pass runtime across configurations. We evaluate these methods under three scaling regimes: (i) fixed model size with increasing number of names processed, (ii) fixed number of names with increasing model size, and (iii) scaling both simultaneously.

We do not directly compare performance between our CUDA implementation and *microgpt*, as the pure Python implementation is prohibitively slow (our CUDA implementation is over $10,000\times$ faster on a medium benchmark). However, we use *microgpt* to validate correctness by comparing logits from a single-sequence forward pass against those produced by the original implementation.

For analyzing results, we will abbreviate methods as UT = unbatched_torch, BT = batched_torch, SC = serial_cpp, and PC = parallel_cpp. We abbreviate presets as S, M, L, VL, and XL for the standard size sweep, and MS5, MM5, ML5, MVL5, and MXL5 for the 5k-step model sweep.

We begin by examining the case of a fixed model size with increasing input name counts, shown in Figure 1 below.

Our PC method outperforms all other baselines across every input size benchmarked. For both SC and UT, this result is unsurprising, as PC is more heavily optimized for this specific workload. More notably, PC also outperforms BT, which is a highly optimized and widely used ML framework. This advantage can largely be attributed to Python overhead in BT, including costs associated with object management and data movement between Python and underlying implementations.

However, a closer examination of the results shows that, while PC currently outperforms BT, its runtime increases more rapidly with input size, whereas BT exhibits near-ideal scaling behavior. It is therefore reasonable to expect that as input sizes continue to grow, particularly to those typical of modern ML workloads, PC will eventually lose its performance advantage.

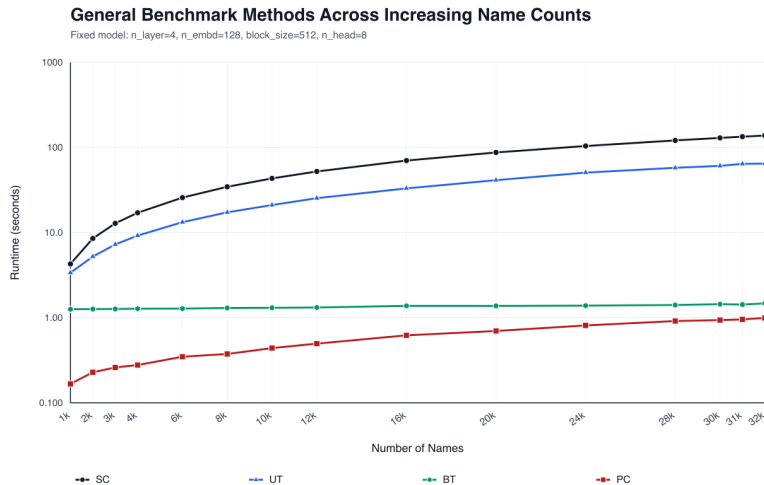


Figure 1: Results from Input Scaling

Next we will look the results from both our other scaling approaches: holding input size constant while increasing the model size ref, and increasing both the input size and model size ref. The model presets both of these experiments are presents below in table 2

Preset	n_layer	n_embd	block_size	n_head	steps
S	1	64	128	4	200
M	2	128	128	8	1000
L	4	256	256	8	2500
VL	6	384	512	12	5000
XL	8	512	512	16	10000
MS5	1	64	512	4	5000
MM5	2	128	512	8	5000
ML5	4	256	512	8	5000
MVL5	6	384	512	12	5000
MXL5	8	512	512	16	5000

Table 2: Model Configurations Used in Experiments

Preset	UT	BT	SC	PC	Preset	UT	BT	SC	PC
S	1.04	0.89	0.06	0.11	MS5	4.33	0.92	1.23	0.20
	0.06×	0.07×	1.00×	0.58×		0.28×	1.34×	1.00×	6.22×
M	2.17	1.05	2.16	0.15	MM5	6.81	1.10	10.76	0.26
	0.99×	2.06×	1.00×	14.46×		1.58×	9.80×	1.00×	41.38×
L	7.36	2.33	46.56	0.37	ML5	12.45	2.37	93.53	0.50
	6.33×	19.98×	1.00×	125.21×		7.51×	39.45×	1.00×	185.50×
VL	20.10	5.78	327.49	1.04	MVL5	20.20	5.79	327.28	1.07
	16.29×	56.69×	1.00×	315.81×		16.21×	56.50×	1.00×	307.03×
XL	51.34	12.51	1579.67	3.51	MXL5	32.29	12.34	784.64	2.10
	30.77×	126.27×	1.00×	450.23×		24.30×	63.59×	1.00×	373.60×

Table 3: Runtime (top) and speedup (bottom) per method. **Speedup is relative to SC**

5 Bottleneck Discussion

To perform the bottleneck analysis, we utilized nvprof to perform a kernel-level profiling of our program. What we wanted to discover was what portions or kernel calls were responsible the largest portion of program time. Below is the configuration for the model and training set size used in the profiling. The batch size parameter is only applicable to the second profiling result, after optimizations were introduced.

Parameter	Value
Names	10,000
Batch Size	512
Number of Layers	8
Embedding Dimension	512
Block Size	64
Number of Heads	16

Table 4: Profiling Configuration Parameters

Kernel	Time (%)	Total Time	Calls	Avg	Max
linear_kernel	80.86%	70.3209 s	490000	143.51 μ s	464.80 μ s
rmsnorm_kernel	10.33%	8.98660 s	170000	52.862 μ s	82.272 μ s
self_attn_kernel	7.33%	6.37602 s	80000	79.700 μ s	162.91 μ s
cross_entropy_loss_kernel	0.85%	736.82 ms	10000	73.682 μ s	149.54 μ s
add_vec_kernel	0.25%	217.70 ms	160000	1.3600 μ s	1.9520 μ s
relu_kernel	0.13%	109.93 ms	80000	1.3740 μ s	2.1440 μ s
embedding_lookup_kernel	0.02%	16.086 ms	10000	1.6080 μ s	2.1760 μ s

Table 5: CUDA Kernel Profiling Results for Parallel Implementation Pre-Optimizations

What we noticed in the above table were two things:

1. Over 80% of execution time was spent in the `linear_kernel`.
2. Individual kernel calls were reasonably fast, but the high number of invocations led to significant overhead.

From these investigations, we determined the right course of action was to pursue optimizations similar to how modern day LLM trainings operate. The first optimziation we went to implement was an optimization to reduce the time cost of the linear layer, which is simply matrix multiplication. Utilizing the tiling method we learned about in lecture, we were able to reduce the time cost of the `linear_kernel` by roughly 81.87x in the average case. The next optimization we wanted to aim for was one that many modern training systems do today, which is batching. By processing single tokens or single sequences at a time, we lose the benefit of the massive amounts of parallelization available to us via the GPUs, and as such implementing the method to perform multiple sequences, at once, in a single batch was a reasonable next step.

Kernel	Time (%)	Total Time	Calls	Avg	Max
linear_kernel	85.88%	2.40485 s	1372	1.7528 ms	6.9377 ms
rmsnorm_kernel	4.52%	126.59 ms	476	265.95 μ s	424.22 μ s
self_attn_kernel	4.45%	124.61 ms	224	556.29 μ s	1.1746 ms
cross_entropy_loss_kernel	3.73%	104.36 ms	28	3.7271 ms	6.7604 ms
relu_kernel	0.58%	16.380 ms	224	73.123 μ s	133.66 μ s
add_vec_kernel	0.42%	11.762 ms	448	26.254 μ s	50.016 μ s
embedding_lookup_kernel	0.02%	481.57 μ s	28	17.198 μ s	31.776 μ s

Table 6: CUDA Kernel Profiling Results for Parallel Implementation with Optimizations

6 Lessons Learned

Placeholder text.

References

- [1] Andrej Karpathy. microgpt, February 2026. Blog post. <https://karpathy.github.io/2026/02/12/microgpt/>.
- [2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, February 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.